

My Week 3 Collaborative Filtering Movie Recommender

Student report by Petr Vorozhtsov

My project: <https://github.com/PetrVorozhtsov/RecSys-LLMs>

My commit: 066577b

My dataset: MovieLens 100K - 943 users, 1,682 movies, and 100,000 ratings

1. What I did

I completed the Week 3 collaborative filtering assignment. I reused the MovieLens 100K data from Week 2, but changed the algorithm from content-based genre matching to rating-pattern collaborative filtering. I implemented both User-Based CF and Item-Based CF with Top-5 results.

2. How I represented the data

I loaded u.item and u.data from the project. I kept movie titles for display and built a user-item rating matrix indexed by the original user IDs and movie IDs. A missing rating is stored as 0 because real ratings are from 1 to 5.

3. How my recommender works

For user-based recommendations, I compare the active user with every other user using cosine similarity over co-rated movies only, keep the Top-20 neighbors, and predict unseen movies with a weighted average.

For item-based recommendations, I compare candidate movie columns with movies the active user already rated, then aggregate similarities weighted by the user rating.

For ranking, I remove already rated movies, sort by predicted score, and display five results.

4. What I changed in my code

File	What I changed
data.js	I load MovieLens ratings and build the user-item rating matrix.
script.js	I implemented cosine similarity, User-Based CF, Item-Based CF, rendering, and analysis.
index.html	I added one user selector, two result panels, and a business analysis section.
style.css	I kept the Week 2 visual style and separated the two CF methods.
readme.md	I documented how I run and evaluate my solution.

5. My experiment - Scenario A

I selected User 1 as the active user. This user has 272 ratings, so both methods had enough history to compare patterns and produce full Top-5 lists.

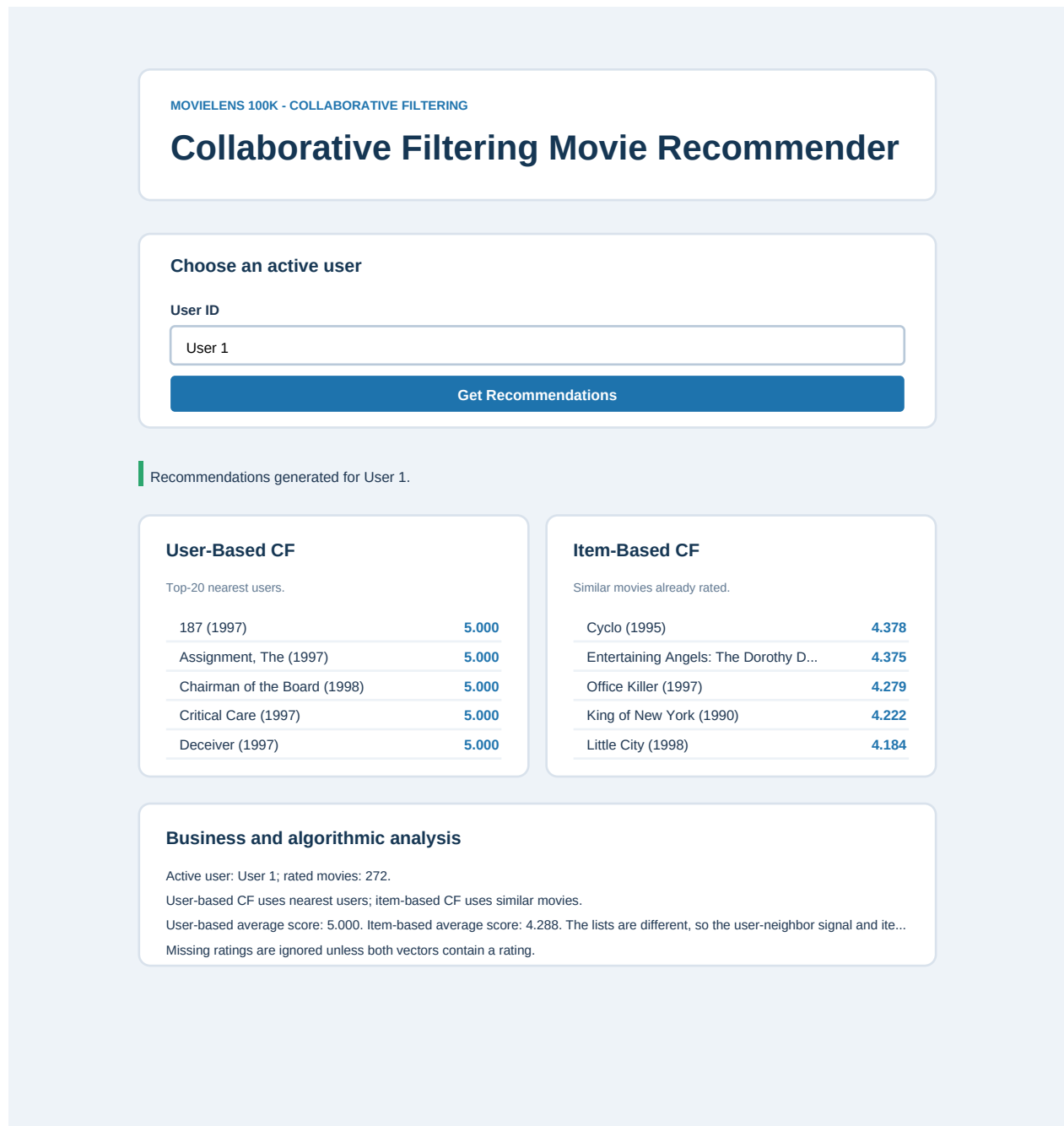


Figure 1. My Scenario A collaborative filtering result summary.

6. My experiment - Scenario B

I selected User 196 as the active user. This user has 39 ratings, so the smaller profile made the sparse-rating problem more visible.

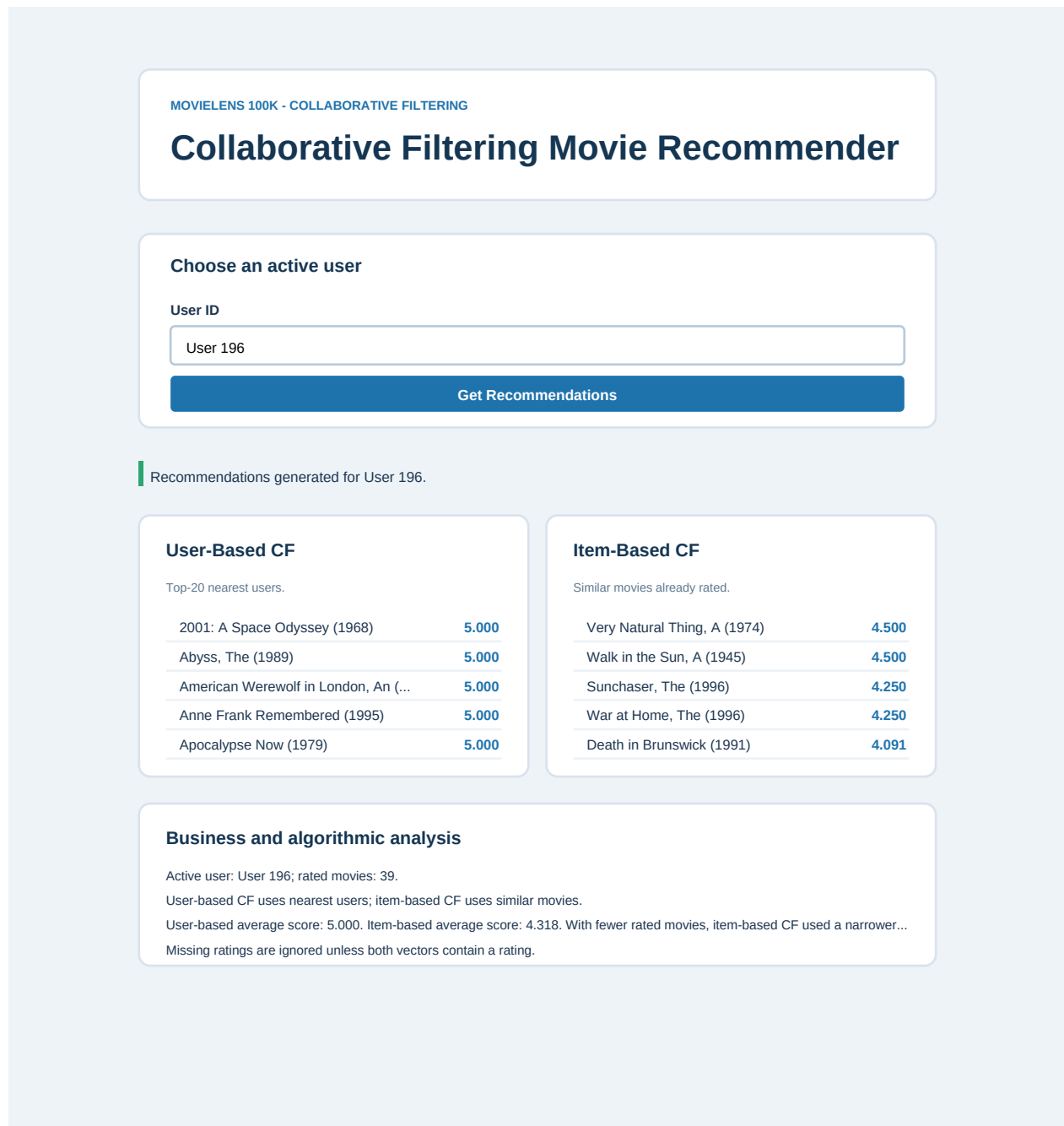


Figure 2. My Scenario B collaborative filtering result summary.

7. My analysis and conclusion

I learned that User-Based CF and Item-Based CF answer the same recommendation question with different evidence. User-Based CF looks for similar people, so it reacts directly to the selected user neighborhood. Item-Based CF looks for similar movies, so it can be more stable and easier to cache when many users reuse the same item similarities. In this dataset there are 943 users and 1,682 movies, so both methods are small enough to run in the browser, but item-based similarities would be more reusable in a larger service.

For missing values, I chose co-rated entries only. This is simple and avoids treating unknown ratings as dislikes. Mean imputation would make vectors denser but can bias users toward average taste. A weighted common-rating approach can reduce unreliable similarities from tiny overlaps. Matrix factorization can discover latent patterns, but it costs more to train and explain.

I also observed the cold-start limitation. Collaborative filtering needs rating history. A new user with no ratings or a new movie with no ratings cannot get reliable similarity scores. Sparse overlaps are also risky because two users or two movies can look similar from too little evidence.

8. How I ran my project

I ran `cd week3` and then `python -m http.server 8000`. I opened `http://localhost:8000/`, selected User 1 and User 196, and clicked Get Recommendations.

My student project: <https://github.com/PetrVorozhtsov/RecSys-LLMs>